

C DEEPAK
202411028
MTECH(VLSI)

DIGITAL PROGRAMMING(EL-467)

PROJECT ON

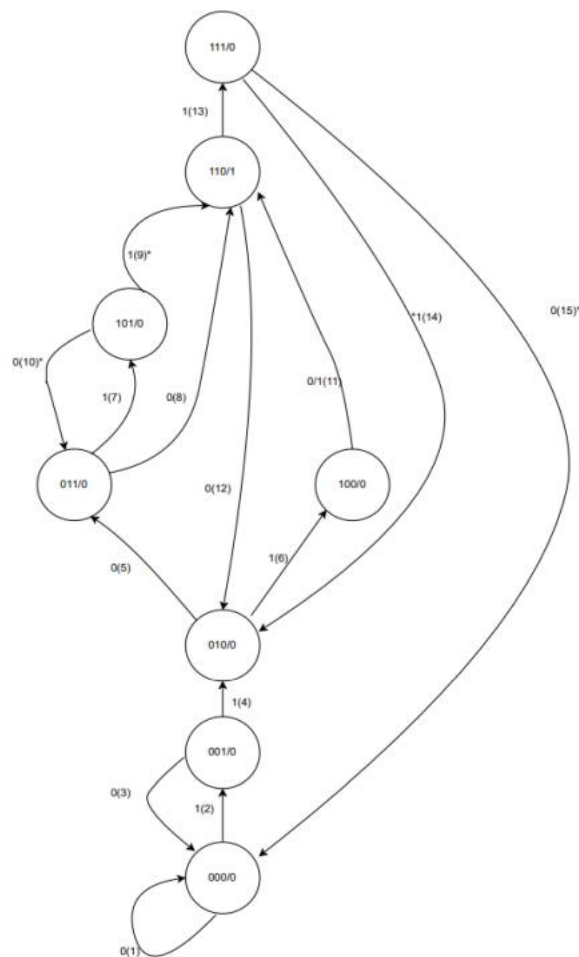
“Design and Implementation of Automated Teller Machine(ATM) Using Finite State Machine (FSM) Controller”.

Under the guidance of
Dr. Yash Agarwal
Professor

Introduction:

Automated Teller Machine is a device which is used for money transaction without the need to visit the bank for such transactions. This application is very useful when we have to just deposit or withdraw small amount of cash. FSM consists of state in which a particular task is performed which can be unique or same as another state. We approach this problem by designing a Finite State Machine (FSM) according to the requirements provided by the establishment. Then we proceed to code the ATM accordingly with FSM designed in system Verilog language. The Finite State Machine designed will control the whole ATM machine by identifying inputs and authenticating them. Then it provides the output signal for valid input statements which enables the ATM machine to work in the optimal and secure manner.

State Diagram:



The above diagram represents the Moore state machine FSM which is used to implement the ATM Controller Machine. A Moore FSM is an state machine where transition of states happen based on input values. The output is defined in the state itself and it is independent of input value. The different states and their implementations are as follows:

State 0: It is represented as 000 and has an output of 0. This state functions as a main screen display which is seen in any ATM. When input is logic '0' it remains in same state and when input is logic '1' it moves to the next state.

- State 1: It is represented as 001 and has an output of 0. This state functions as a Pin entry and verification display to verify the customer. When input is logic '1' it goes to next state and when input is logic '0' it moves to the previous state.
- State 2: It is represented as 010 and has an output of 0. This state functions as a screen display where customer can choose to either deposit or withdraw. When input is logic '0' it goes to state 3 and when input is logic '1' it moves to the state 4.
- State 3: It is represented as 011 and has an output of 0. If customer chooses to withdraw he enters this state. When input is logic '0' it goes to state 5 and when input is logic '1' it moves to the state 6.
- State 4: It is represented as 100 and has an output of 0. If customer chooses to deposit he enters this state. When input is logic '0' it goes to state 6 and when input is logic '1' it moves to the state 6 as well.
- State 5: It is represented as 101 and has an output of 0. If customer has to perform face recognition process he enters this state. When input is logic '0' it goes to state 3 and when input is logic '1' it moves to the state 6.
- State 6: It is represented as 110 and has an output of 1. In this state the balance of customers account is updated according to his transaction mode. When input is logic '0' it goes to state 2 and when input is logic '1' it moves to the state 7.
- State 7: It is represented as 111 and has an output of 0. Here the customer receives a statement of his transaction and can choose to remove card or make another transaction. When input is logic '0' it goes to state 0 and when input is logic '1' it moves to the state 2.

Code:

```
`define true 1'b1
`define false 1'b0
`define FIND 1'b0
`define AUTHENTICATE 1'b1
`define WAITING      3'b000
`define MENU         3'b010
`define BALANCE       3'b011
`define WITHDRAW      3'b100
`define WITHDRAW_SHOW_BALANCE 3'b101
`define TRANSACTION   3'b110
```

```
module authentication(
    input [11:0] accNumber,
    input [3:0] pin,
    input action,
    input deAuth,
    output reg wasSuccessful,
    output reg [3:0] accIndex
);
```

```
reg [11:0] acc_database [0:9];
reg [3:0] pin_database [0:9];
```

```
//initializing the database with arbitrary accounts
```

```
initial begin
```

```
    acc_database[0] = 12'd2749; pin_database[0] = 4'b0000;
```

```
    acc_database[1] = 12'd2175; pin_database[1] = 4'b0001;
```

```
acc_database[2] = 12'd2429; pin_database[2] = 4'b0010;
acc_database[3] = 12'd2125; pin_database[3] = 4'b0011;
acc_database[4] = 12'd2178; pin_database[4] = 4'b0100;
acc_database[5] = 12'd2647; pin_database[5] = 4'b0101;
acc_database[6] = 12'd2816; pin_database[6] = 4'b0110;
acc_database[7] = 12'd2910; pin_database[7] = 4'b0111;
acc_database[8] = 12'd2299; pin_database[8] = 4'b1000;
acc_database[9] = 12'd2689; pin_database[9] = 4'b1001;
end
```

```
always @ (deAuth) begin
```

```
    if(deAuth == `true)
```

```
        wasSuccessful = 1'bx;
```

```
end
```

```
//looping through the database
```

```
// if action is set to find then it'll simply try to find a match for the given accNumber and
returns its index
```

```
integer i;
```

```
always @(accNumber or pin) begin
```

```
    wasSuccessful = `false;
```

```
    accIndex = 0;
```

```
//loop through the data base
```

```
for(i = 0; i < 10; i = i+1) begin
```

```
    //found a match for accNumber
```

```
    if(accNumber == acc_database[i]) begin
```

```
        if(action == `FIND) begin
```

```
            wasSuccessful = `true;
```

```
            accIndex = i;
```

```

        end

        if(action == `AUTHENTICATE) begin
            if(pin == pin_database[i]) begin
                wasSuccessful = `true;
                accIndex = i;

            end
        end
    end
end

endmodule

//

module ATM(
    input clk,
    input exit,
    input [11:0] accNumber,
    input [3:0] pin,
    input [11:0] destinationAcc,
    input [2:0] menuOption,
    input [10:0] amount,
    output reg error,
    output reg [10:0] balance
);

```

```
//initializing the balance database with an arbitrary amount of money
```

```
reg [15:0] balance_database [0:9];
```

```
initial begin
```

```
    $display("Welcome to the ATM");
```

```
    balance_database[0] = 16'd500;
```

```
    balance_database[1] = 16'd500;
```

```
    balance_database[2] = 16'd500;
```

```
    balance_database[3] = 16'd500;
```

```
    balance_database[4] = 16'd500;
```

```
    balance_database[5] = 16'd500;
```

```
    balance_database[6] = 16'd500;
```

```
    balance_database[7] = 16'd500;
```

```
    balance_database[8] = 16'd500;
```

```
    balance_database[9] = 16'd500;
```

```
end
```

```
reg [2:0] currState = `WAITING;
```

```
wire [3:0] accIndex;
```

```
wire [3:0] destinationAccIndex;
```

```
wire isAuthenticated;
```

```
wire wasFound;
```

```
reg deAuth = `false;
```

```
authentication authAccNumberModule(accNumber, pin, `AUTHENTICATE, deAuth,  
isAuthenticated, accIndex);
```

```
authentication findAccNumberModule(destinationAcc, 0, `FIND, deAuth, wasFound,  
destinationAccIndex);
```

```
//main block of module with asynchronous exit
```

```
always @(posedge clk or isAuthenticated or menuOption or exit)
```

```
begin
```

```
    //restart the error
```

```
        error = `false;
```

```
    if(exit == `true)
```

```
    begin
```

```
        //transition to the waiting state
```

```
        currState = `WAITING;
```

```
        //deauthenticate the current user
```

```
        deAuth = `true;
```

```
        #20;
```

```
    end
```

```
    if(currState == `MENU)
```

```
    begin
```

```
        //set the selected option as the current state
```

```
        if((menuOption >= 0) & (menuOption <= 7))
```

```
        begin
```

```
            currState = menuOption;
```

```
        end else
```

```
            currState = menuOption;
```

```
    end
```

```
    //switch case for the menu options
```

```
    //the rest is pretty straight forward
```

```
    case (currState)
```



```
` WAITING: begin  
  
if (isAuthenticated == ` true) begin  
    currState = ` MENU;  
    $display("Logged In.");  
end  
  
else if(isAuthenticated == ` false) begin  
    $display("Account number or password was incorrect");  
    currState = ` WAITING;  
end  
end
```

```
` BALANCE: begin  
  
balance = balance_database[acclIndex];  
  
$display("Account %d has balance %d", accNumber, balance_database[acclIndex]);  
  
currState = ` MENU;  
end
```

```
` WITHDRAW: begin  
  
if (amount <= balance_database[acclIndex]) begin  
    balance_database[acclIndex] = balance_database[acclIndex] - amount;  
    balance = balance_database[acclIndex];  
    currState = ` MENU;  
    error = ` false;  
end  
  
else begin  
    currState = ` MENU;
```

```
    error = `true;
end
end
```

```
`WITHDRAW_SHOW_BALANCE: begin
    if (amount <= balance_database[acclIndex]) begin
        balance_database[acclIndex] = balance_database[acclIndex] - amount;
        balance = balance_database[acclIndex];
        currState = `MENU;
        error = `false;

        $display("Account %d has balance %d after withdrawing %d", accNumber,
balance_database[acclIndex], amount);
    end
    else begin
        currState = `MENU;
        error = `true;
    end
end
```

```
`TRANSACTION: begin
    if ((amount <= balance_database[acclIndex]) & (wasFound == `true) &
(balance_database[acclIndex] + amount < 2048)) begin
        currState = `MENU;
        error = `false;

        balance_database[destinationAcclIndex] =
balance_database[destinationAcclIndex] + amount;

        balance_database[acclIndex] = balance_database[acclIndex] - amount;

        $display("Destination account %d after transaction has a total balance of %d",
destinationAcc, balance_database[destinationAcclIndex]);
    end
end
```

```
        else begin
            currState = `MENU;
            error = `true;
        end
    end
endcase
end
endmodule
```

Testbench:

```
`define true 1'b1
`define false 1'b0
`define FIND 1'b0
`define AUTHENTICATE 1'b1
`define WAITING      3'b000
`define GET_PIN      3'b001
`define MENU         3'b010
`define BALANCE      3'b011
`define WITHDRAW     3'b100
`define WITHDRAW_SHOW_BALANCE 3'b101
`define TRANSACTION  3'b110
`define DONE         3'b111

module atm_tb();
    reg clk, exit;
    reg [11:0] accNumber;
    reg [3:0] pin;
    reg [11:0] destinationAccNumber;
    reg [2:0] menuOption;
```

```
reg [10:0] amount;  
wire error;  
wire [10:0] balance;
```

```
ATM atmModule(clk, exit, accNumber, pin, destinationAccNumber, menuOption,  
amount, error, balance);
```

```
initial begin  
    clk = 1'b0;  
end
```

```
always @(error) begin  
    if(error == `true)  
        $display("Error!, action causes an invalid operation.");  
end
```

```
initial begin  
    //incorrect PIN  
    accNumber = 12'd2278;  
    pin = 4'b0100;
```

```
    //valid credentials  
#30 accNumber = 12'd2178;  
    pin = 4'b0100;
```

```
#30  
    //withdraw some money and then show the balance  
#30 amount = 100;  
    menuOption = `WITHDRAW_SHOW_BALANCE;  
    clk = ~clk;#5clk = ~clk;
```

#30

//show the balance

 #30 menuOption = ` BALANCE;

clk = ~clk;#5clk = ~clk;

#30

//withdraw too much money, resulting in an error

amount = 2500;

 menuOption = ` WITHDRAW;

clk = ~clk;#5clk = ~clk;

//the balance wont change because an error happened during withdrawal

 #30 menuOption = ` BALANCE;

clk = ~clk;#5clk = ~clk;

#30

//transfer some money to the destination account with number 2816

#30 amount = 50;

destinationAccNumber = 2816;

 menuOption = ` TRANSACTION;

clk = ~clk;#5clk = ~clk;

#30

//transfer too much money to the destination account with number 2816 which exceeds 2047 and cuases an error

amount = 2550;

destinationAccNumber = 2816;

 menuOption = ` TRANSACTION;

clk = ~clk;#5clk = ~clk;

#30

//exit the system

exit = 1;

#30

```

exit = 0;

#30

//log in using the account with number 2816

accNumber = 12'd2816;

pin = 4'b0110;

#30

//you'll see that the balance is more than the default value because we had
transferred some money to this account a while ago

#30;

menuOption = `BALANCE;

clk = ~clk;#5clk = ~clk;

end

endmodule

```

Waveform:

The state diagram is implemented on vivado software and we get output as this simulation. User input is the data or information that is entered into a system by the user it is used to determine what output the system will produce.

